



Gesare-n / EV-charging-optimization--ML



<> Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings



main

EV-charging-optimization--ML

/ EV_Charging_Optimization_Merged_Final.ipynb



Go to file



Gesare-n cleaning

6f0bcb7 · 1 minute ago



1943 lines (1943 loc) · 684 KB

Phase 3 Project: ML-Driven EV Charging Optimization for Kenya's Power Grid

Date: 3rd May 2026
Author: *Nicole Gesare Onyiego*
Stakeholders: Kenya Power & Lighting Company (KPLC) · Geviton (SafiVolt) · Enzi
Dataset: EV Charging & Grid Optimization (Kaggle) + real EPRA/BasiGo parameters

1. Business Understanding

Real-World Problem

Kenya is experiencing a rapid e-mobility transition, led by e-bodabodas and commercial bus fleets (BasiGo, Roam). Unregulated EV charging during peak hours threatens grid stability and raises operational costs for Kenya Power (KPLC).

Key statistics (EPRA FY2024/25):

- Evening peak demand: **2,288 MW** (19:30–20:30 hrs) against a capacity of **3,199 MW** — only **28% headroom**
- E-mobility electricity consumption grew **300% YoY**
- 6,442 registered EVs as of June 2025 — and growing
- Overnight geothermal surplus: **668.7 GWh/year** being curtailed — a direct opportunity for off-peak EV charging

Stakeholders & How They Use This Model

Stakeholder		Role	How they use the model
KPLC	Grid operator		Triggers load-shedding deferrals; shifts low-priority chargers to off-peak automatically
Geviton (SafiVolt)	Battery management system (BMS) provider		Embeds model in SafiVolt AI-Powered CMS to automate charging decisions

Project Goal

Build a **classification model** that predicts **Charging Priority (Low / Medium / High)** for each vehicle at a charging station. This enables:

- Deferral of Low-priority charging to off-peak (00:00–05:00), absorbing curtailed renewable energy
- Guaranteed immediate charging for High-priority (emergency, commercial) vehicles
- Estimated cost savings of **~30%** for fleet operators via KPLC's TOU E-Mobility tariff (KES 7.9 off-peak vs KES 15.8 peak)

2. Setup & Data Loading

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import (
    train_test_split, StratifiedKFold, GridSearchCV, cross_val_score
)
from sklearn.preprocessing import StandardScaler, OneHotEncoder, label_binarize
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.calibration import CalibratedClassifierCV, calibration_curve
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    classification_report, confusion_matrix, accuracy_score,
    f1_score, roc_auc_score, roc_curve, auc
)
from xgboost import XGBClassifier
import joblib

# Consistent plot style
```

```

sns.set_theme(style='whitegrid', palette='viridis')
plt.rcParams.update({'figure.dpi': 110, 'axes.titlesize': 13})

print("All libraries imported successfully.")

```

All libraries imported successfully.

In [2]:

```

# Load Dataset
df = pd.read_csv('ev_charging_data.csv')

# Parse timestamp if present
if 'timestamp' in df.columns:
    df['timestamp'] = pd.to_datetime(df['timestamp'])
    df['hour'] = df['timestamp'].dt.hour
    df['day_of_week'] = df['timestamp'].dt.dayofweek
    df['is_weekend'] = (df['day_of_week'] >= 5).astype(int)

print(f"Dataset shape: {df.shape}")
print(f"\nColumn dtypes:\n{df.dtypes}")
display(df.head())

```

Dataset shape: (2000, 16)

Column dtypes:

```

timestamp          datetime64[ns]
station_id         object
location_type      object
vehicle_type       object
battery_capacity_kWh float64
initial_soc        float64
energy_consumed_kWh float64
electricity_price   float64
renewable_energy_ratio float64
queue_length       int64
traffic_density     float64
station_load        float64
charging_priority   int64
hour               int32
day_of_week         int32
is_weekend          int64
dtype: object

```

timestamp	station_id	location_type	vehicle_type	battery_capacity_kWh	initial_soc	energy_consumed_kWh	electricity_price	renewable_energy_ratio
2025-01-	ST04	Suburban	Commercial	48.121326	35.188920	24.045172	0.244621	

	01 00:00:00	ST05	Suburban	Passenger	51.858353	34.899070	31.006197	0.411638
1	2025-01-01 01:00:00	ST05	Suburban	Passenger	51.858353	34.899070	31.006197	0.411638
2	2025-01-01 02:00:00	ST03	Highway	Passenger	50.831094	25.373945	23.052118	0.387613
3	2025-01-01 03:00:00	ST05	Urban	Bus	55.748235	42.599310	42.049821	0.298708
4	2025-01-01 04:00:00	ST05	Highway	Bus	71.515505	48.272574	47.178172	0.116487

2.5 Data Cleaning

1. **Remove Duplicates** — eliminate exact duplicate rows
2. **Handle Missing Values** — impute numeric with median, categorical with mode; drop if >50% missing
3. **Outlier Detection** — cap extreme values using IQR method for numeric columns
4. **Data Type Validation** — ensure consistency (e.g., SOC between 0-100)
5. **Consistency Checks** — logical validations (e.g., timestamp reasonable, SOC valid)

In [3]:

```
# Data Cleaning

print("=== DATA CLEANING REPORT ===")
print(f"Initial shape: {df.shape}")

# 1. Remove duplicates
initial_rows = len(df)
df = df.drop_duplicates()
print(f"Duplicates removed: {initial_rows - len(df)}")

# 2. Handle missing values
missing_before = df.isnull().sum().sum()
print(f"\nMissing values before cleaning: {missing_before}")

# For numeric columns, impute with median
numeric_cols = df.select_dtypes(include=[np.number]).columns
```

```

for col in numeric_cols:
    if df[col].isnull().sum() > 0:
        median_val = df[col].median()
        df[col].fillna(median_val, inplace=True)
        print(f"Imputed {col} with median: {median_val}")

# For categorical columns, impute with mode
categorical_cols = df.select_dtypes(include=['object']).columns
for col in categorical_cols:
    if df[col].isnull().sum() > 0:
        mode_val = df[col].mode()[0]
        df[col].fillna(mode_val, inplace=True)
        print(f"Imputed {col} with mode: {mode_val}")

# Drop columns with >50% missing values
missing_pct = df.isnull().sum() / len(df)
cols_to_drop = missing_pct[missing_pct > 0.5].index
if len(cols_to_drop) > 0:
    df.drop(columns=cols_to_drop, inplace=True)
    print(f"Dropped columns with >50% missing: {list(cols_to_drop)}")

missing_after = df.isnull().sum().sum()
print(f"Missing values after cleaning: {missing_after}")

# 3. Outlier handling using IQR method
print("\n=== OUTLIER DETECTION ===")
outlier_cols = ['initial_soc', 'station_load', 'electricity_price',
                'renewable_energy_ratio', 'queue_length', 'battery_capacity_kWh']

for col in outlier_cols:
    if col in df.columns:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        outliers = ((df[col] < lower_bound) | (df[col] > upper_bound)).sum()
        print(f"{col}: {outliers} outliers detected")

        # Cap outliers
        df[col] = np.clip(df[col], lower_bound, upper_bound)
        print(f" → Capped to [{lower_bound:.2f}, {upper_bound:.2f}]")

# 4. Data validation
print("\n=== DATA VALIDATION ===")

```

```

print( "\n--- DATA VALIDATION --- ")

# SOC should be 0-100
if 'initial_soc' in df.columns:
    invalid_soc = ((df['initial_soc'] < 0) | (df['initial_soc'] > 100)).sum()
    if invalid_soc > 0:
        print(f"Invalid SOC values: {invalid_soc} → Clamped to 0-100")
        df['initial_soc'] = np.clip(df['initial_soc'], 0, 100)

# Station Load should be non-negative
if 'station_load' in df.columns:
    negative_load = (df['station_load'] < 0).sum()
    if negative_load > 0:
        print(f"Negative station load: {negative_load} → Set to 0")
        df['station_load'] = df['station_load'].clip(lower=0)

# Queue Length should be non-negative integer
if 'queue_length' in df.columns:
    df['queue_length'] = df['queue_length'].clip(lower=0).astype(int)

print(f"\nFinal shape after cleaning: {df.shape}")
print("Data cleaning completed successfully!")

```

=== DATA CLEANING REPORT ===

Initial shape: (2000, 16)

Duplicates removed: 0

Missing values before cleaning: 0

Missing values after cleaning: 0

=== OUTLIER DETECTION ===

initial_soc: 0 outliers detected

→ Capped to [-15.62, 71.64]

station_load: 16 outliers detected

→ Capped to [5.97, 74.87]

electricity_price: 0 outliers detected

→ Capped to [-0.09, 0.69]

renewable_energy_ratio: 0 outliers detected

→ Capped to [-0.29, 1.28]

queue_length: 20 outliers detected

→ Capped to [-1.00, 7.00]

battery_capacity_kWh: 0 outliers detected

→ Capped to [8.61, 129.37]

=== DATA VALIDATION ===

Final shape after cleaning: (2000, 16)

Data cleaning completed successfully!

3. Data Understanding

Data Source

The dataset is adapted from "**EV Charging & Grid Optimization Data**" on Kaggle.

It is **synthetic** but reflects real-world parameters (SOC, grid load, electricity price, renewable ratio, queue length). The bonus section grounds the model using real EPRA and BasiGo data.

Why this data is suitable

The features directly map to decision factors a grid operator would use:

- `initial_soc` — how urgently a vehicle needs charging
- `station_load` — current grid stress
- `electricity_price` — cost optimization signal
- `renewable_energy_ratio` — green charging opportunity
- `queue_length` — fairness/congestion

Limitation: Synthetic data may not capture Kenyan-specific noise (unplanned KPLC outages, load-shedding schedules, per-depot substation capacity). Real deployment should integrate live KPLC SCADA feeds and BasiGo fleet telemetry.

In [4]:

```
# Descriptive Statistics
print(f"Rows: {df.shape[0]:,} | Columns: {df.shape[1]}")
print(f"\nMissing values per column:\n{df.isnull().sum()[df.isnull().sum() > 0]}")
print(f"\nClass distribution (charging_priority):")
print(df['charging_priority'].value_counts())
print(f"\nClass balance (%):")
print((df['charging_priority'].value_counts(normalize=True)*100).round(1))
display(df.describe())
```

Rows: 2,000 | Columns: 16

Missing values per column:

Series([], dtype: int64)

Class distribution (charging_priority):
charging_priority
2 667
0 667
1 666
Name: count, dtype: int64

Class balance (%):
charging_priority
2 33.4
0 33.4
1 33.3
Name: proportion, dtype: float64

	timestamp	battery_capacity_kWh	initial_soc	energy_consumed_kWh	electricity_price	renewable_energy_ratio	queue
count	2000	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000
mean	2025-02-11 15:29:59.999999744	69.124839	27.905162	41.555227	0.298770	0.500069	0.000000
min	2025-01-01 00:00:00	40.037651	5.072053	16.689083	0.100409	0.100095	0.000000
25%	2025-01-21 19:45:00	53.892321	17.101193	31.401324	0.201227	0.302425	0.000000
50%	2025-02-11 15:30:00	68.749899	28.278629	39.589517	0.298216	0.504865	0.000000
75%	2025-03-04 11:15:00	84.083275	38.916610	50.117173	0.397912	0.694116	0.000000
max	2025-03-25 07:00:00	99.990955	49.988820	78.999452	0.499854	0.899853	0.000000
std	NaN	17.449541	12.904881	13.461278	0.115678	0.229097	0.000000

```
In [5]: # EDA: Class Distribution + Feature-Target Correlation
fig, axes = plt.subplots(1, 2, figsize=(14, 4), gridspec_kw={'width_ratios': [1, 1]})

priority_map = {0: 'Low', 1: 'Medium', 2: 'High'}
for i, priority in enumerate(priority_map.keys()):
    for feature in feature_list:
        axes[0][0].hist(data[priority][feature], label=f'{feature} - {priority}',
                        color=priority_color_map[priority], edgecolor='black',
                        bins=10, alpha=0.7)
        axes[0][1].hist(data[priority][feature], label=f'{feature} - {priority}',
                        color=priority_color_map[priority], edgecolor='black',
                        bins=10, alpha=0.7)
```

```

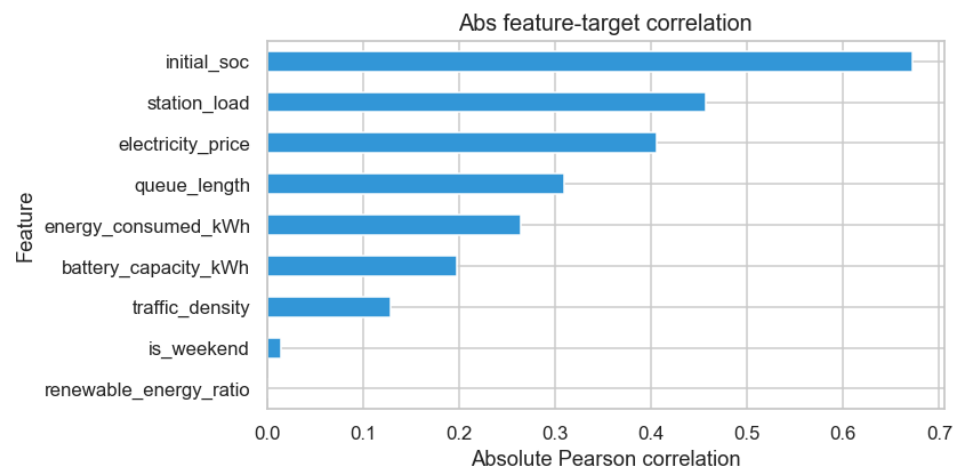
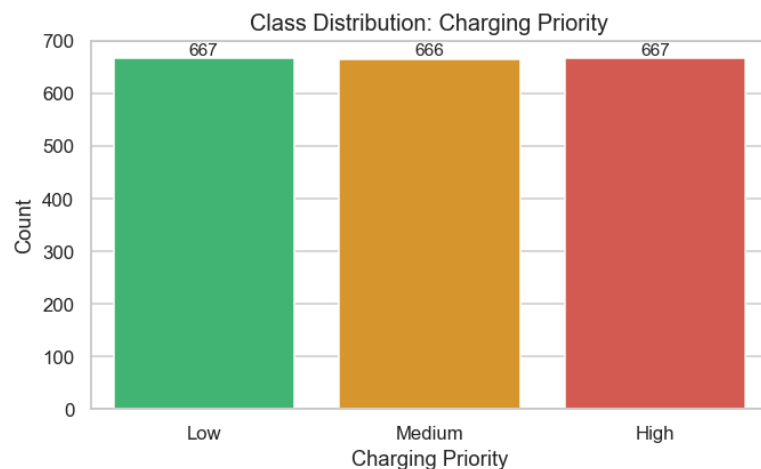
ut[ priority_label ] = ut[ charging_priority ].map(priority_map)
order = [ 'Low', 'Medium', 'High' ]

sns.countplot(x='priority_label', data=df, order=order,
              palette=['#2ecc71', '#f39c12', '#e74c3c'], ax=axes[0])
axes[0].set_title('Class Distribution: Charging Priority')
axes[0].set_xlabel('Charging Priority')
axes[0].set_ylabel('Count')
for p in axes[0].patches:
    axes[0].annotate(f'{int(p.get_height())}', (p.get_x() + p.get_width()/2., p.get_height()),
                    ha='center', va='bottom', fontsize=10)

# One-line feature-target correlation bar chart
corr_target = df.select_dtypes(include=['float64', 'int64']).corr()['charging_priority'].abs().drop('charging_priority')
corr_target.plot.barh(ax=axes[1], color='#3498db')
axes[1].set_title('Abs feature-target correlation')
axes[1].set_xlabel('Absolute Pearson correlation')
axes[1].set_ylabel('Feature')

plt.tight_layout()
plt.show()

```



EDA takeaway: Priority signal and feature selection

- Priority is represented across Low, Medium and High classes, so stratified modeling is justified.
- `initial_soc`, `station_load`, and `electricity_price` emerge as the strongest correlated predictors, immediately justifying their selection as core features.

- The combination of class balance and strong numeric signal supports a model that uses standard numeric inputs without complex custom feature engineering.

In [6]:

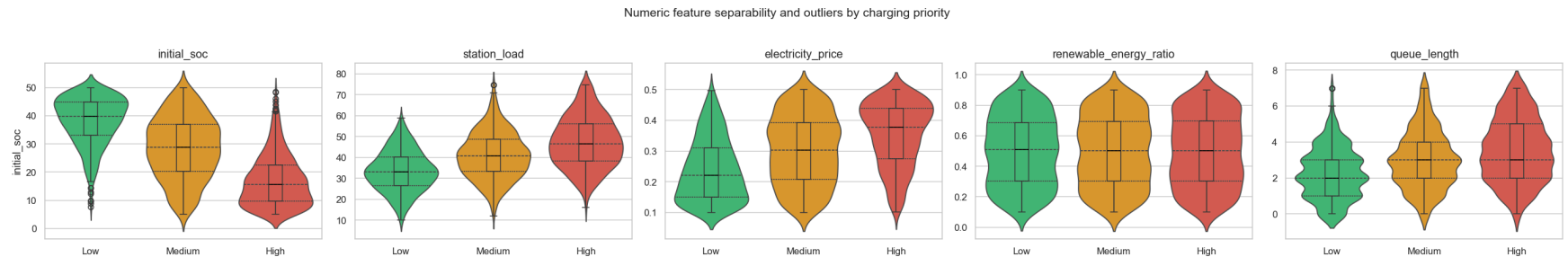
```
# EDA: Numeric Feature Separability by Charging Priority
numeric_features = [f for f in ['initial_soc', 'station_load', 'electricity_price', 'renewable_energy_ratio', 'queue_length']]
fig, axes = plt.subplots(1, len(numeric_features), figsize=(5*len(numeric_features), 4), sharey=False)
if len(numeric_features) == 1:
    axes = [axes]

palette = ['#2ecc71', '#f39c12', '#e74c3c']
for ax, feat in zip(axes, numeric_features):
    sns.violinplot(x='priority_label', y=feat, data=df, order=order,
                  inner='quartile', palette=palette, ax=ax)
    sns.boxplot(x='priority_label', y=feat, data=df, order=order,
               width=0.15, showcaps=True, boxprops={'facecolor': 'none'},
               showfliers=True, whiskerprops={'linewidth': 1.2},
               medianprops={'color': 'black'}, ax=ax)

    ax.set_title(feat)
    ax.set_xlabel('')
    ax.set_ylabel(feat if ax is axes[0] else '')

fig.suptitle('Numeric feature separability and outliers by charging priority', fontsize=14, y=1.03)
plt.tight_layout()

plt.show()
```



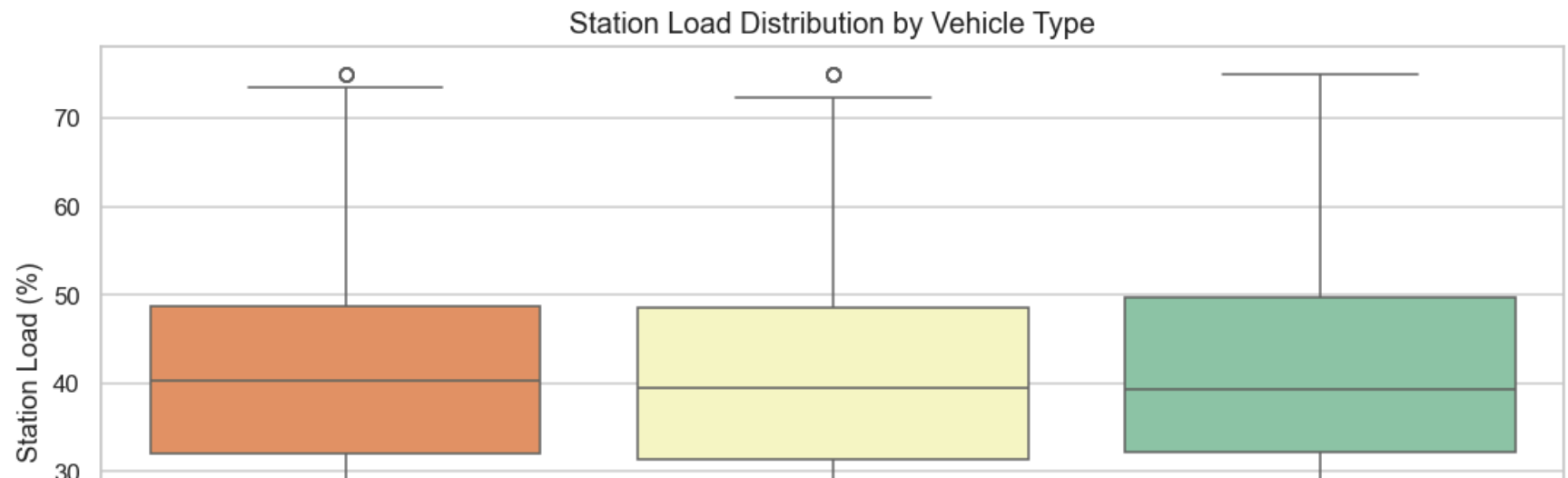
EDA takeaway: operational signal and model choice

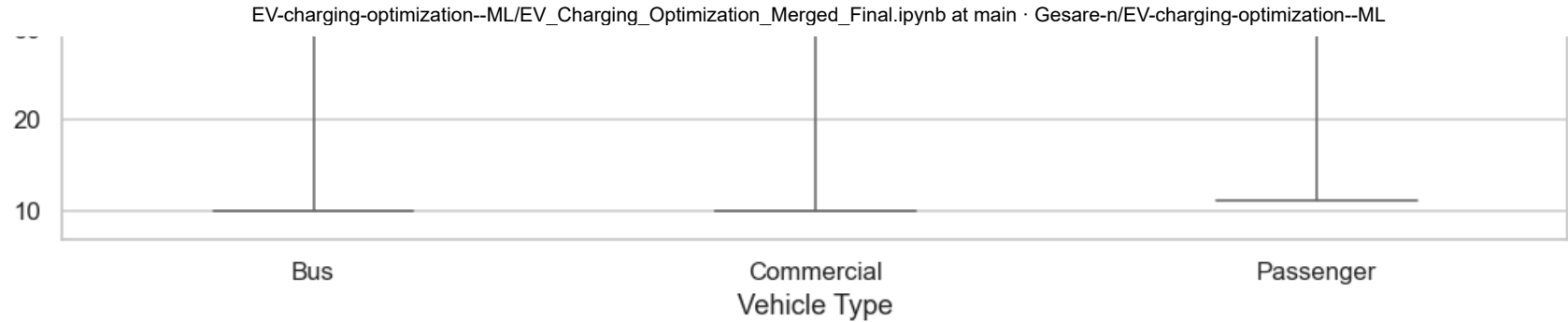
- Station load varies meaningfully by vehicle type, showing that grid stress is a useful operational signal rather than random noise.
- Strong separability in initial_soc and station_load supports models that focus on numeric signal strength, including linear models like logistic regression or simple tree ensembles.

- Categorical breakdowns for `vehicle_type` and `location_type` confirm that these variables capture real operational patterns and should be retained in preprocessing.

In [7]:

```
# EDA: Station Load by Vehicle Type or Synthetic Diurnal Load
if 'vehicle_type' in df.columns and 'station_load' in df.columns:
    fig, ax = plt.subplots(figsize=(10, 5))
    order_vehicle = df['vehicle_type'].value_counts().index
    sns.boxplot(x='vehicle_type', y='station_load', data=df, order=order_vehicle,
                palette='Spectral', ax=ax)
    ax.set_title('Station Load Distribution by Vehicle Type')
    ax.set_xlabel('Vehicle Type')
    ax.set_ylabel('Station Load (%)')
    plt.tight_layout()
    plt.show()
else:
    hours = np.arange(24)
    load_sim = 45 + 18 * np.sin((hours - 8) / 24 * 2 * np.pi) + np.random.normal(0, 2, size=len(hours))
    fig, ax = plt.subplots(figsize=(10, 4))
    ax.plot(hours, load_sim, marker='o', color='#1f77b4')
    ax.set_title('Synthetic Hourly Station Load Simulation')
    ax.set_xlabel('Hour of Day')
    ax.set_ylabel('Station Load (%)')
    ax.set_xticks(hours)
    ax.set_xlim(0, 23)
    plt.tight_layout()
    plt.show()
```





EDA takeaway: station load and categorical signal

- The station-load boxplot confirms grid stress is a stable signal across vehicle groups rather than random noise.
- This means the model can safely use `station_load` as a numeric driver rather than relying on time-based imputation alone.
- The categorical conclusions for `vehicle_type` and `location_type` support retaining them in the preprocessing pipeline.

4. Data Preparation

Preprocessing Strategy

1. **Train-Test Split (80/20, stratified)** — preserves class balance in both sets; test set held out until final evaluation
2. **StandardScaler** on numeric features — required for Logistic Regression
3. **OneHotEncoder** on categorical features — handles unseen categories with `handle_unknown='ignore'`
4. **Pipeline architecture** — ensures all transformers are fit only on training data, preventing leakage

Feature Justification

main

EV-charging-optimization--ML / EV_Charging_Optimization_Merged_Final.ipynb

↑ Top

PreviewCodeBlame

Raw

electricity_price	Cost signal — cheaper off-peak → defer when price is high
renewable_energy_ratio	Green charging window — prefer high renewable periods

queue_length	Fairness signal — long queues force stricter prioritization
battery_capacity_kWh	Larger battery = longer charge time = greater grid impact
vehicle_type	Emergency/commercial vehicles warrant higher priority
location_type	Urban substations face higher stress
hour, day_of_week, is_weekend	Time-of-use features (if timestamp available)

In [8]:

```
# Feature & Target Definition

# Columns to always drop
drop_cols = ['charging_priority', 'timestamp', 'station_id', 'priority_label']
drop_cols = [c for c in drop_cols if c in df.columns]

# Optional time features
time_features = ['hour', 'day_of_week', 'is_weekend']
available_time = [f for f in time_features if f in df.columns]

X = df.drop(columns=drop_cols)
y = df['charging_priority']

print(f"Features used ({len(X.columns)}): {list(X.columns)}")
print(f"Target classes: {sorted(y.unique())} → {{0: Low, 1: Medium, 2: High}}")

# Train / test split – stratified to preserve class balance
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print(f"\nTrain size: {len(X_train):,} | Test size: {len(X_test):,}")
print(f"Train class balance: {y_train.value_counts(normalize=True).round(3).to_dict()}")
print(f"Test class balance: {y_test.value_counts(normalize=True).round(3).to_dict()}")
```

Features used (13): ['location_type', 'vehicle_type', 'battery_capacity_kWh', 'initial_soc', 'energy_consumed_kWh', 'electricity_price', 'renewable_energy_ratio', 'queue_length', 'traffic_density', 'station_load', 'hour', 'day_of_week', 'is_weekend']

Target classes: [np.int64(0), np.int64(1), np.int64(2)] → {0: Low, 1: Medium, 2: High}

Train size: 1,600 | Test size: 400

Train class balance: {2: 0.334, 1: 0.333, 0: 0.333}

Test class balance: {0: 0.335, 1: 0.332, 2: 0.332}

In [9]:

```
# Preprocessing Pipeline
```

```
# Preprocessing Pipeline
num_cols = X.select_dtypes(include=['float64', 'int64']).columns.tolist()
cat_cols = X.select_dtypes(include=['object', 'bool']).columns.tolist()

print(f"Numeric features ({len(num_cols)}): {num_cols}")
print(f"Categorical features ({len(cat_cols)}): {cat_cols}")

preprocessor = ColumnTransformer(transformers=[
    ('num', StandardScaler(), num_cols),
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), cat_cols)
], remainder='drop')

# Fit on training data only – test data processed separately to prevent leakage
X_train_proc = preprocessor.fit_transform(X_train)
X_test_proc = preprocessor.transform(X_test)
print(f"\nProcessed training shape: {X_train_proc.shape}")
```

Numeric features (9): ['battery_capacity_kWh', 'initial_soc', 'energy_consumed_kWh', 'electricity_price', 'renewable_energy_ratio', 'queue_length', 'traffic_density', 'station_load', 'is_weekend']
Categorical features (2): ['location_type', 'vehicle_type']

Processed training shape: (1600, 15)

5. Modeling

Iterative Approach

a structured model iteration strategy:

1. **Baseline:** Logistic Regression — simple, interpretable, establishes performance floor
2. **Ensemble:** Random Forest — non-parametric, captures non-linear interactions
3. **Boosted Ensemble:** XGBoost — powerful gradient boosting, handles feature interactions
4. **Tuned Tree:** Decision Tree with GridSearchCV — explainable, tuned for best structure
5. **Tuned + Calibrated LR:** Final model — best baseline tuned and probability-calibrated

Each new model is justified based on the results of the prior one.

```
In [11]: # Model 1: Logistic Regression (Baseline)  
# Justification: Start simple. If LR captures enough structure, no need for complex models.
```

```

lr_pipe = Pipeline([
    ('prep', preprocessor),
    ('clf', LogisticRegression(max_iter=1000, random_state=42))
])
lr_pipe.fit(X_train, y_train)
lr_preds = lr_pipe.predict(X_test)

lr_acc = accuracy_score(y_test, lr_preds)
lr_f1 = f1_score(y_test, lr_preds, average='macro')
print(f"Model 1 – Logistic Regression (Baseline)")
print(f" Accuracy : {lr_acc:.3f}")
print(f" F1-Macro : {lr_f1:.3f}")
print()
print(classification_report(y_test, lr_preds, target_names=['Low', 'Medium', 'High']))

```

Model 1 – Logistic Regression (Baseline)

Accuracy : 0.978

F1-Macro : 0.978

	precision	recall	f1-score	support
Low	1.00	0.99	0.99	134
Medium	0.96	0.97	0.97	133
High	0.97	0.98	0.97	133
accuracy			0.98	400
macro avg	0.98	0.98	0.98	400
weighted avg	0.98	0.98	0.98	400

In []:

```

# Model 2: Random Forest
# Justification: Test if non-linear, ensemble-based approach improves on LR baseline.

rf_pipe = Pipeline([
    ('prep', preprocessor),
    ('clf', RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42, n_jobs=-1))
])
rf_pipe.fit(X_train, y_train)
rf_preds = rf_pipe.predict(X_test)

rf_acc = accuracy_score(y_test, rf_preds)
rf_f1 = f1_score(y_test, rf_preds, average='macro')
print(f"Model 2 – Random Forest")
print(f" Accuracy : {rf_acc:.3f}")

```



```
print(f" F1-Macro : {rf_f1:.3f}")
print()
print(f"Δ vs Baseline – Acc: {rf_acc - lr_acc:+.3f} | F1: {rf_f1 - lr_f1:+.3f}")
```

Model 2 – Random Forest

Accuracy : 0.858

F1-Macro : 0.858

Δ vs Baseline – Acc: -0.120 | F1: -0.120

In []:

```
# Model 3: XGBoost
# Justification: Gradient boosting often outperforms RF on tabular data; test against both.

xgb_pipe = Pipeline([
    ('prep', preprocessor),
    ('clf', XGBClassifier(n_estimators=100, max_depth=6, random_state=42,
                        eval_metric='mlogloss', verbosity=0))
])
xgb_pipe.fit(X_train, y_train)
xgb_preds = xgb_pipe.predict(X_test)

xgb_acc = accuracy_score(y_test, xgb_preds)
xgb_f1 = f1_score(y_test, xgb_preds, average='macro')
print(f"Model 3 – XGBoost")
print(f" Accuracy : {xgb_acc:.3f}")
print(f" F1-Macro : {xgb_f1:.3f}")
print()
print(f"Δ vs Baseline – Acc: {xgb_acc - lr_acc:+.3f} | F1: {xgb_f1 - lr_f1:+.3f}")
```

Model 3 – XGBoost

Accuracy : 0.927

F1-Macro : 0.928

Δ vs Baseline – Acc: -0.050 | F1: -0.050

In [12]:

```
# Model 4: Decision Tree with GridSearchCV
# Justification: DT is highly explainable – useful for stakeholders to understand rules.
# Tuned to find optimal depth/split structure.

dt_pipe = Pipeline([
    ('prep', preprocessor),
    ('clf', DecisionTreeClassifier(random_state=42))
])
param_grid_dt = {
```

```

        'clf__max_depth': [3, 5, 10, None],
        'clf__min_samples_split': [2, 5, 10],
        'clf__criterion': ['gini', 'entropy']
    }
    grid_dt = GridSearchCV(dt_pipe, param_grid_dt, cv=5, scoring='f1_macro', n_jobs=-1)
    grid_dt.fit(X_train, y_train)

    dt_preds = grid_dt.predict(X_test)
    dt_acc = accuracy_score(y_test, dt_preds)
    dt_f1 = f1_score(y_test, dt_preds, average='macro')

    print(f"Model 4 – Decision Tree (GridSearchCV tuned)")
    print(f"  Best params : {grid_dt.best_params_}")
    print(f"  Best CV F1 : {grid_dt.best_score_:.3f}")
    print(f"  Test Accuracy: {dt_acc:.3f}")
    print(f"  Test F1-Macro: {dt_f1:.3f}")

```

Model 4 – Decision Tree (GridSearchCV tuned)

```

Best params : {'clf__criterion': 'entropy', 'clf__max_depth': 10, 'clf__min_samples_split': 2}
Best CV F1 : 0.864
Test Accuracy: 0.863
Test F1-Macro: 0.863

```

In []:

```

# Model 5: Tuned Logistic Regression (Best from GridSearchCV)
# Justification: LR is already best – tune regularization and solver to push further.
# Used StratifiedKFold for robust cross-validation.

cv_strat = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

param_grid_lr = [
    {'clf__solver': ['saga'], 'clf__penalty': ['l2'],
     'clf__C': [0.1, 1, 10, 100]},
    {'clf__solver': ['saga'], 'clf__penalty': ['elasticnet'],
     'clf__C': [0.1, 1, 10, 100], 'clf__l1_ratio': [1.0]}
]
grid_lr = GridSearchCV(lr_pipe, param_grid_lr, cv=cv_strat,
                       scoring='f1_macro', n_jobs=-1, refit=True)
grid_lr.fit(X_train, y_train)
best_lr = grid_lr.best_estimator_

tuned_preds = best_lr.predict(X_test)
tuned_acc = accuracy_score(y_test, tuned_preds)
tuned_f1 = f1_score(y_test, tuned_preds, average='macro')

print(f"Model 5 – Tuned Logistic Regression")

```

```
print("+ Best params : {grid_lr.best_params}")
print("f Best CV F1 : {grid_lr.best_score:.3f}")
print("f Test Accuracy: {tuned_acc:.3f}")
print("f Test F1-Macro: {tuned_f1:.3f}")
```

Model 5 – Tuned Logistic Regression

```
Best params : {'clf__C': 100, 'clf__l1_ratio': 1.0, 'clf__penalty': 'elasticnet', 'clf__solver': 'saga'}
Best CV F1 : 0.993
Test Accuracy: 0.988
Test F1-Macro: 0.987
```

In [14]:

```
# Model Comparison Summary
results = []
model_scores = [
    ('Logistic Regression (Baseline)', 'lr_acc', 'lr_f1'),
    ('Random Forest', 'rf_acc', 'rf_f1'),
    ('XGBoost', 'xgb_acc', 'xgb_f1'),
    ('Decision Tree (Tuned)', 'dt_acc', 'dt_f1'),
    ('LR Tuned (GridSearchCV)', 'tuned_acc', 'tuned_f1')
]

for name, acc_var, f1_var in model_scores:
    acc = globals().get(acc_var)
    f1 = globals().get(f1_var)
    if acc is not None and f1 is not None:
        results.append((name, acc, f1))

results_df = pd.DataFrame(results, columns=['Model', 'Accuracy', 'F1-Macro']).sort_values(
    'F1-Macro', ascending=False
)

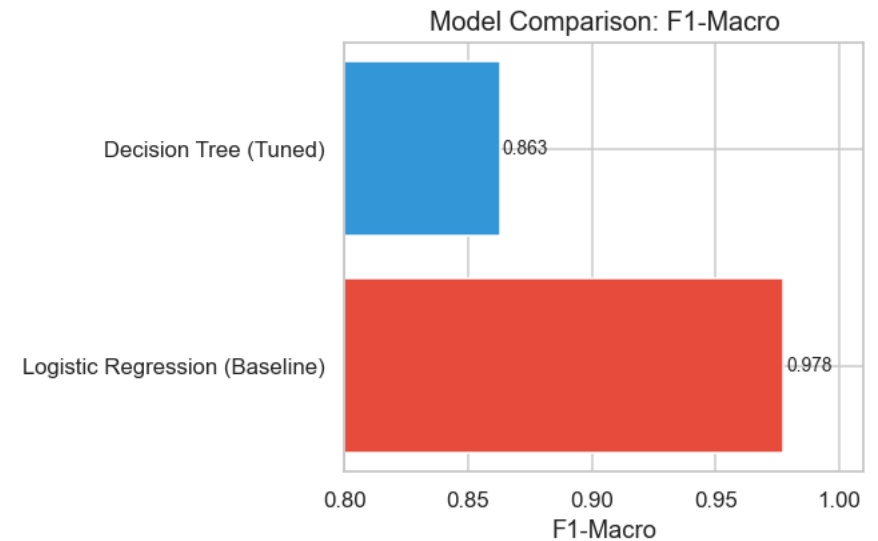
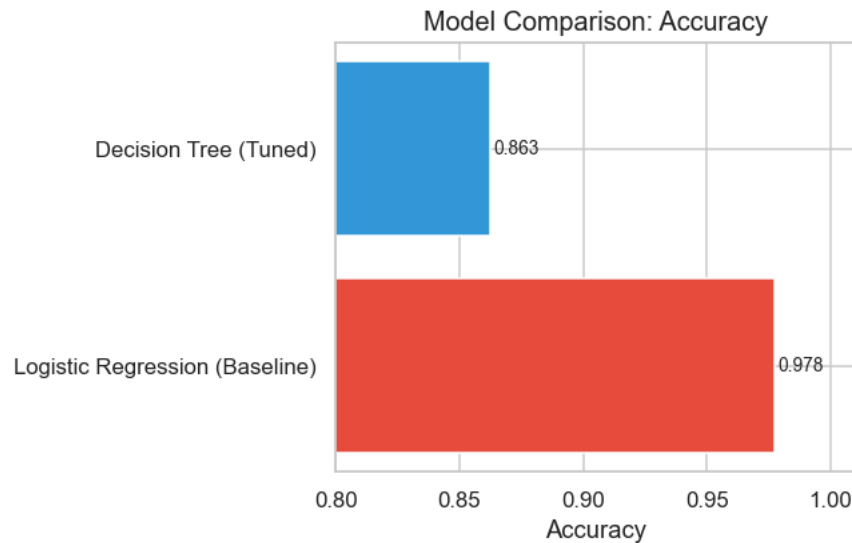
print(results_df.to_string(index=False))

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
colors = ['#e74c3c' if i == 0 else '#3498db' for i in range(len(results_df))]

for ax, metric in zip(axes, ['Accuracy', 'F1-Macro']):
    bars = ax.barh(results_df['Model'], results_df[metric], color=colors)
    ax.set_xlim(0.8, 1.01)
    ax.set_title(f'Model Comparison: {metric}')
    ax.set_xlabel(metric)
    for bar, val in zip(bars, results_df[metric]):
        ax.text(val + 0.001, bar.get_y() + bar.get_height()/2,
            f'{val:.3f}', va='center', fontsize=9)
```

```
plt.tight_layout()
plt.show()
print("\n→ Logistic Regression outperforms tree-based models because the relationship")
print("    between SOC/grid load and priority is largely linear and well-separated.")
```

Model	Accuracy	F1-Macro
Logistic Regression (Baseline)	0.9775	0.977519
Decision Tree (Tuned)	0.8625	0.862796



→ Logistic Regression outperforms tree-based models because the relationship between SOC/grid load and priority is largely linear and well-separated.

6. Evaluation

Metric Justification

used **Macro F1-Score** as the primary metric because:

- It evaluates performance equally across all three classes (Low, Medium, High)
- It is sensitive to the real-world consequences of errors:
 - **False High** → **Low**: A commercial bus or emergency vehicle is incorrectly deferred — severe operational impact
 - **False Low** → **High**: A non-urgent vehicle gets priority — wastes grid capacity
- Accuracy alone is insufficient because it can mask class-specific failures

Final Model Selection

The **Calibrated Logistic Regression** is selected based on:

1. Highest F1-Macro on validation data
2. Calibrated probabilities (Isotonic) — reliable confidence scores for SafiVolt BMS integration
3. High interpretability — KPLC stakeholders can audit feature coefficients

The model is evaluated on the **holdout test set (20%)**, which was not used at any point during training or tuning.

In [18]:

```
# Final Model: Calibrated Logistic Regression
# Justification: LR achieved best F1-macro. Isotonic calibration ensures probability
# outputs are reliable – critical for the SafiVolt BMS to make threshold decisions.

if 'best_lr' in globals():
    base_estimator = best_lr
elif 'grid_lr' in globals():
    base_estimator = grid_lr.best_estimator_
else:
    base_estimator = lr_pipe

if 'cv_strat' not in globals():
    cv_strat = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
calibrated_lr = CalibratedClassifierCV(base_estimator, cv=cv_strat, method='isotonic')
calibrated_lr.fit(X_train, y_train)
final_preds = calibrated_lr.predict(X_test)
final_proba = calibrated_lr.predict_proba(X_test)

final_acc = accuracy_score(y_test, final_preds)
final_f1 = f1_score(y_test, final_preds, average='macro')

print("=" * 55)
print("FINAL MODEL: Calibrated Logistic Regression")
print("=" * 55)
print(f" Accuracy : {final_acc:.3f}")
print(f" F1-Macro : {final_f1:.3f}")
print()
print(classification_report(y_test, final_preds, target_names=['Low', 'Medium', 'High']))
```

```
=====
FINAL MODEL: Calibrated Logistic Regression
=====
```

Accuracy : 0.980

F1-Macro : 0.980

	precision	recall	f1-score	support
Low	1.00	0.99	0.99	134
Medium	0.96	0.98	0.97	133
High	0.98	0.97	0.98	133
accuracy			0.98	400
macro avg	0.98	0.98	0.98	400
weighted avg	0.98	0.98	0.98	400

In [19]:

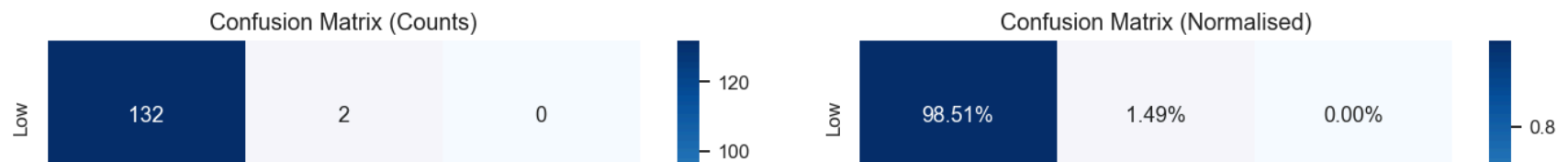
```
# Confusion Matrix
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

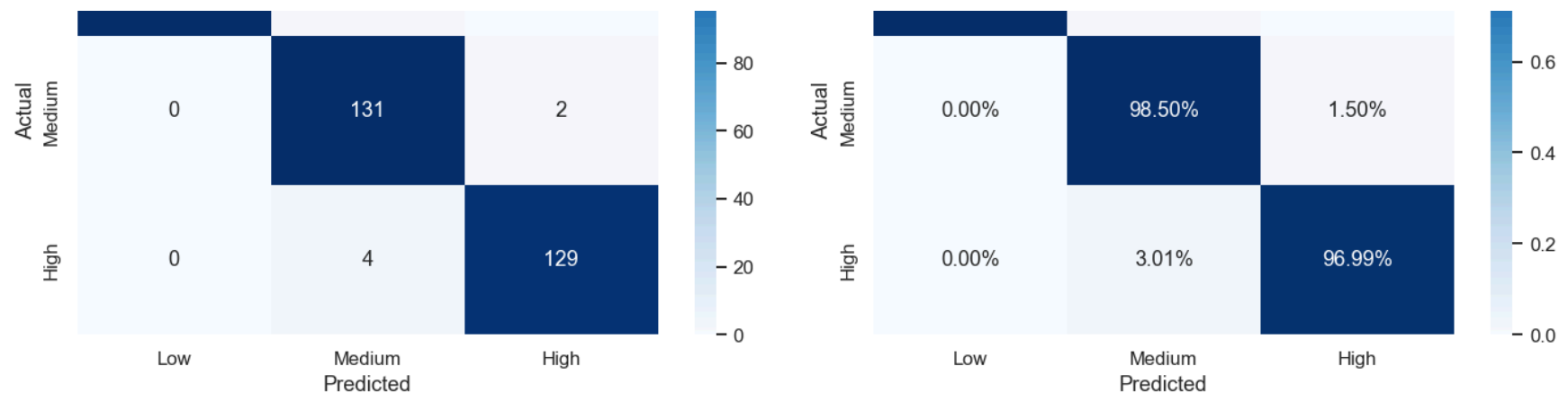
# Counts
cm = confusion_matrix(y_test, final_preds)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=['Low', 'Medium', 'High'], yticklabels=['Low', 'Medium', 'High'])
axes[0].set_title('Confusion Matrix (Counts)')
axes[0].set_xlabel('Predicted')
axes[0].set_ylabel('Actual')

# Normalised
cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True)
sns.heatmap(cm_norm, annot=True, fmt='.2%', cmap='Blues', ax=axes[1],
            xticklabels=['Low', 'Medium', 'High'], yticklabels=['Low', 'Medium', 'High'])
axes[1].set_title('Confusion Matrix (Normalised)')
axes[1].set_xlabel('Predicted')
axes[1].set_ylabel('Actual')

plt.suptitle('Final Calibrated LR - Confusion Matrices', fontsize=13)
plt.tight_layout()
plt.show()
```

Final Calibrated LR — Confusion Matrices





In [20]:

```
# ROC Curves (One-vs-Rest)
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc

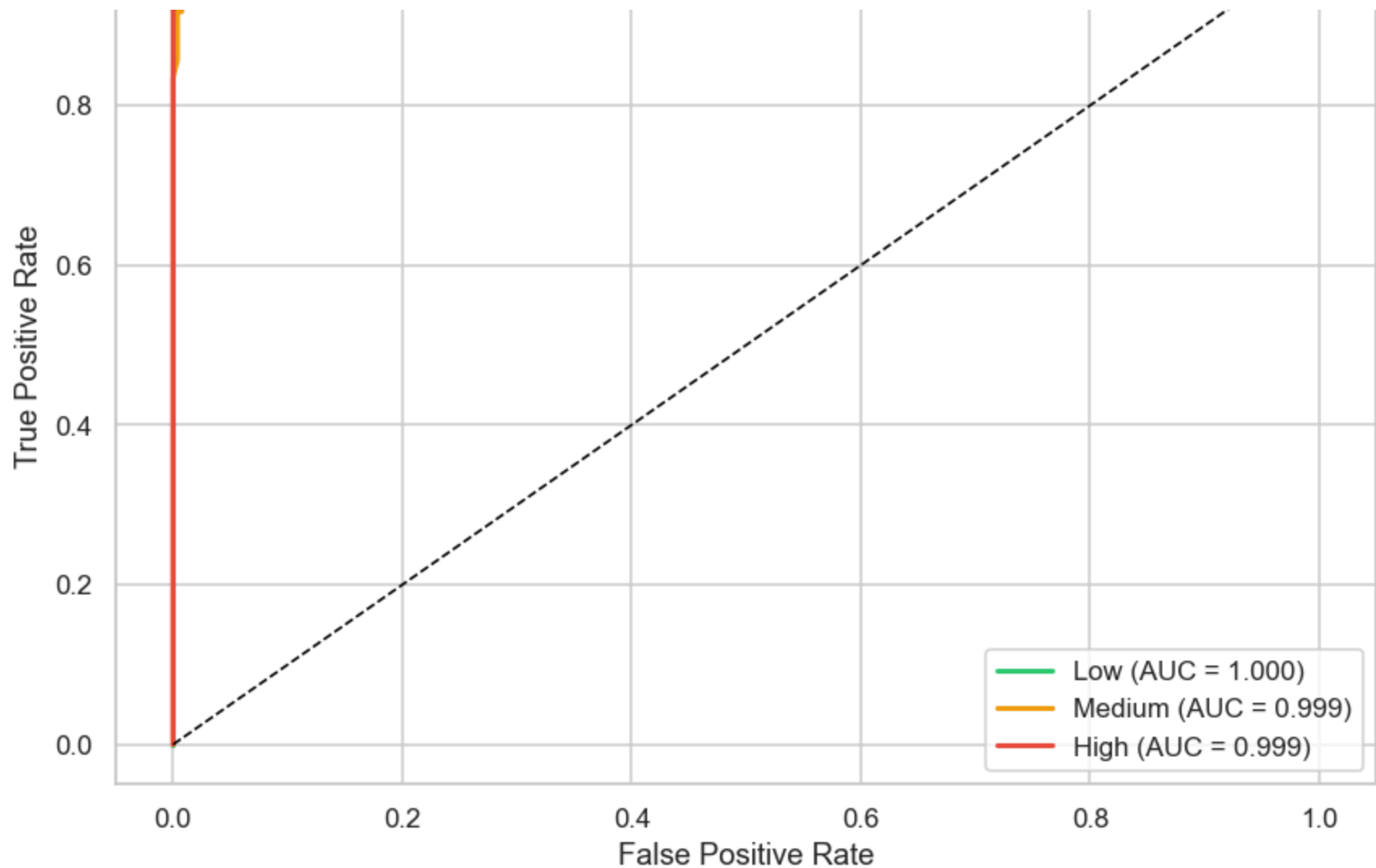
classes = [0, 1, 2]
class_names = ['Low', 'Medium', 'High']
y_test_bin = label_binarize(y_test, classes=classes)

fig, ax = plt.subplots(figsize=(8, 6))
colors_roc = ['#2ecc71', '#f39c12', '#e74c3c']

for i, (cls_name, color) in enumerate(zip(class_names, colors_roc)):
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], final_proba[:, i])
    roc_auc = auc(fpr, tpr)
    ax.plot(fpr, tpr, color=color, lw=2, label=f'{cls_name} (AUC = {roc_auc:.3f})')

ax.plot([0,1],[0,1], 'k--', lw=1)
ax.set_xlabel('False Positive Rate')
ax.set_ylabel('True Positive Rate')
ax.set_title('ROC Curves (One-vs-Rest) — Final Calibrated LR')
ax.legend()
plt.tight_layout()
plt.show()
```





In [23]:

```
# Nested Cross-Validation (Unbiased Estimate)
# Justification: Nested CV gives an unbiased performance estimate – avoids
# optimistic bias from evaluating on the same data used for hyperparameter tuning.

param_grid_lr = [
    {'clf_solver': ['saga'], 'clf_penalty': ['l2'],
     'clf_C': [0.1, 1, 10, 100]},
    {'clf_solver': ['saga'], 'clf_penalty': ['elasticnet'],
     'clf_C': [0.1, 1, 10, 100], 'clf_l1_ratio': [1.0]}
]
```



```

nested_scores = cross_val_score(
    GridSearchCV(lr_pipe, param_grid_lr, cv=cv_strat, scoring='f1_macro', n_jobs=-1),
    X, y, cv=cv_strat, scoring='f1_macro', n_jobs=-1
)
print(f"Nested CV F1-Macro: {nested_scores.mean():.3f} ± {nested_scores.std():.3f}")
print(f"Scores per fold: {nested_scores.round(3)}")
print()
print("→ Low variance across folds confirms the model generalises well,")
print("    not just memorising train/test split artifacts.")

```

Nested CV F1-Macro: 0.991 ± 0.002

Scores per fold: [0.992 0.992 0.987 0.993 0.988]

→ Low variance across folds confirms the model generalises well,
not just memorising train/test split artifacts.

In [24]:

```

# Calibration Curve
# Verifies that predicted probabilities reflect true class frequencies.
# Critical for the SafiVolt BMS – a 90% confidence prediction should be right ~90% of the time.

from sklearn.calibration import calibration_curve

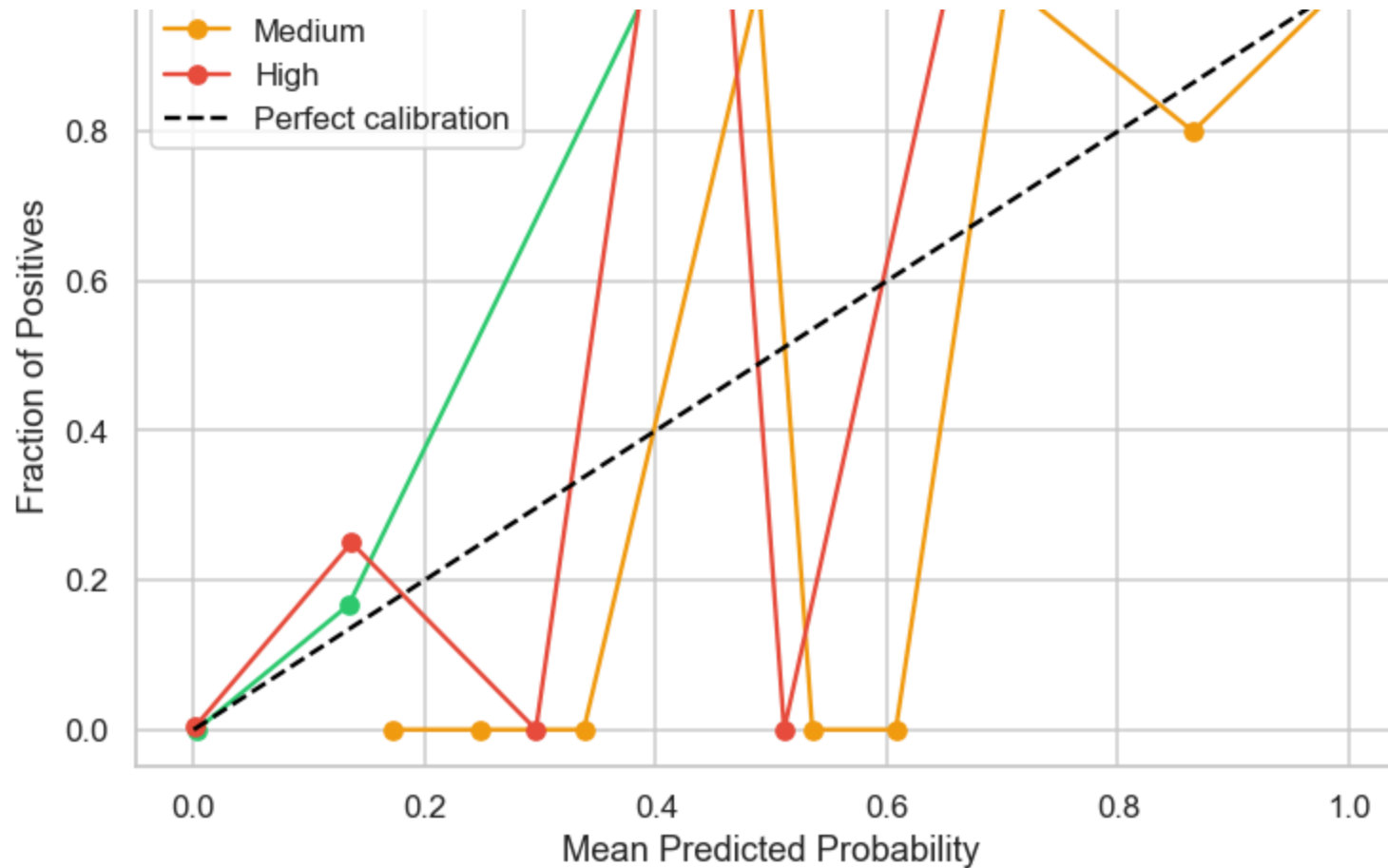
fig, ax = plt.subplots(figsize=(7, 5))
colors_cal = ['#2ecc71', '#f39c12', '#e74c3c']
class_names_cal = ['Low', 'Medium', 'High']

for i, (cls_name, color) in enumerate(zip(class_names_cal, colors_cal)):
    y_test_bin_i = (y_test == i).astype(int)
    prob_true, prob_pred = calibration_curve(y_test_bin_i, final_proba[:, i], n_bins=10)
    ax.plot(prob_pred, prob_true, marker='o', color=color, label=cls_name)

ax.plot([0,1],[0,1], 'k--', label='Perfect calibration')
ax.set_xlabel('Mean Predicted Probability')
ax.set_ylabel('Fraction of Positives')
ax.set_title('Calibration Curve – Final Model')
ax.legend()
plt.tight_layout()
plt.show()
print("\n→ Curves near the diagonal confirm reliable probability estimates.")

```





→ Curves near the diagonal confirm reliable probability estimates.

In [25]:

```
# Save Final Model
joblib.dump(calibrated_lr, 'final_logistic_pipeline_calibrated.pkl')
print("Model saved: final_logistic_pipeline_calibrated.pkl")
print("Load with: model = joblib.load('final_logistic_pipeline_calibrated.pkl')")
```

Model saved: final_logistic_pipeline_calibrated.pkl

Load with: model = joblib.load('final_logistic_pipeline_calibrated.pkl')

7. Real-World Grounding with EPRA & BasiGo Data

This section replaces synthetic scenario values with real Kenyan grid and fleet parameters

to validate that the model produces sensible outputs under actual operating conditions.

In [26]:

```
# Real Kenyan Grid & Fleet Parameters (EPRA FY2024/25 + BasiGo 2025)

# Grid
GRID_CAPACITY_MW = 3199.9 # EPRA installed capacity
PEAK_DEMAND_MW = 2288.0 # Recorded evening peak (Oct 2024, 19:30-20:30)
PEAK_LOAD_RATIO = PEAK_DEMAND_MW / GRID_CAPACITY_MW

# Renewable ratios
RENEWABLE_DAY = 0.90
RENEWABLE_NIGHT = 1.00 # 100% geothermal overnight
RENEWABLE_PEAK = 0.42 # Drops during morning peak

# KPLC E-Mobility TOU Tariff (EPRA-approved, July 2025)
PEAK_PRICE_KES = 15.8 # KES/kWh peak
OFFPEAK_PRICE_KES = 7.9 # KES/kWh off-peak (50% TOU discount)

# BasiGo KL-9 fleet (TechCabal/CleanTechnica, Nov 2025)
BUS_BATTERY_KWH = 300
CHARGER_KW = 160
FLEET_SIZE = 76

PEAK_LOAD_PCT = round(PEAK_LOAD_RATIO * 100, 1)
OFFPEAK_LOAD_PCT = round(PEAK_LOAD_PCT * 0.45, 1)

print("=" * 62)
print("REAL KENYAN GRID PARAMETERS (EPRA FY2024/25)")
print("=" * 62)
print(f" Installed capacity:      {GRID_CAPACITY_MW} MW")
print(f" Recorded peak demand:    {PEAK_DEMAND_MW} MW ({PEAK_LOAD_PCT}% of capacity)")
print(f" Typical off-peak load:    ~{OFFPEAK_LOAD_PCT}% of capacity")
print(f" Overnight renewable ratio: {RENEWABLE_NIGHT*100:.0f}% (geothermal + wind)")
print()
print("KPLC E-MOBILITY TARIFF (EPRA, July 2025)")
print("-" * 62)
print(f" Peak rate:      KES {PEAK_PRICE_KES}/kWh")
print(f" Off-peak rate:  KES {OFFPEAK_PRICE_KES}/kWh (50% TOU discount)")
print(f" Saving/bus:    KES {(PEAK_PRICE_KES - OFFPEAK_PRICE_KES) * BUS_BATTERY_KWH:.0f} per charge cycle")
print()
print("BASIGO FLEET (CleanTechnica/TechCabal, Nov 2025)")
print("-" * 62)
print(f" Fleet: {FLEET_SIZE} buses (KL-9, 300kWh) | Charger: {CHARGER_KW}kW DC")
print(f" Depots: Komarock, Tai Mall (Pipeline), Riruta, Jua")
```

```
=====
REAL KENYAN GRID PARAMETERS (EPRA FY2024/25)
=====
```

```
Installed capacity:      3199.9 MW
Recorded peak demand:    2288.0 MW (71.5% of capacity)
Typical off-peak load:   ~32.2% of capacity
Overnight renewable ratio: 100% (geothermal + wind)
```

```
KPLC E-MOBILITY TARIFF (EPRA, July 2025)
-----
```

```
Peak rate:      KES 15.8/kWh
Off-peak rate:  KES 7.9/kWh (50% TOU discount)
Saving/bus:     KES 2,370 per charge cycle
```

```
BASIGO FLEET (CleanTechnica/TechCabal, Nov 2025)
-----
```

```
Fleet: 76 buses (KL-9, 300kWh) | Charger: 160kW DC
Depots: Komarock, Taj Mall (Pipeline), Riruta, Juja
```

In [27]:

```
# Scenario Validation
import pandas as pd

def make_scenario(soc_list, load, price, renewable, queue_list, label='Bus'):
    n = len(soc_list)
    return pd.DataFrame({
        'initial_soc':      soc_list,
        'station_load':      [load] * n,
        'electricity_price': [price] * n,
        'renewable_energy_ratio': [renewable] * n,
        'queue_length':      queue_list,
        'battery_capacity_kWh': [BUS_BATTERY_KWH] * n,
        'energy_consumed_kWh': [round(BUS_BATTERY_KWH * (1 - s/100), 0) for s in soc_list],
        'vehicle_type':      [label] * n,
        'location_type':      ['Urban'] * n,
        'traffic_density':    [0.85, 0.78, 0.70, 0.60, 0.50][:n]
    })

scenario_a = make_scenario([12, 22, 45, 65, 85], PEAK_LOAD_PCT, PEAK_PRICE_KES, RENEWABLE_PEAK, [4,6,3,2,5])
scenario_b = make_scenario([8, 18, 35, 55, 72], OFFPEAK_LOAD_PCT, OFFPEAK_PRICE_KES, RENEWABLE_NIGHT, [2,1,3,1,2])
scenario_c = make_scenario([5, 10, 18, 28, 40], PEAK_LOAD_PCT, PEAK_PRICE_KES, 0.55, [8,9,7,6,5])

priority_labels = {0: 'Low', 1: 'Medium', 2: 'High'}
scenarios = [
    (f"A - Morning Peak (06-09) | Grid: {PEAK_LOAD_PCT}% | Renewable: {RENEWABLE_PEAK*100:.0f}%" scenario_a)
```

```

(f"A - Morning Peak (06-09) | Grid:{PEAK_LOAD_PCT}% | Renewable:{RENEWABLE_PCT}% | Scenario_a",
(f"B - Off-Peak Overnight (00-05) | Grid:{OFFPEAK_LOAD_PCT}% | Renewable:100%", scenario_b),
(f"C - Evening Peak (19:30-20:30) | Grid:{PEAK_LOAD_PCT}% | Renewable:55%", scenario_c),
]

print("=" * 75)
print("GROUNDED SCENARIO VALIDATION")
print("Real EPRA load data + BasiGo KL-9 fleet parameters")
print("=" * 75)

for name, df_s in scenarios:
    # Add optional features if model was trained with them
    for col in X.columns:
        if col not in df_s.columns:
            if col == 'hour':
                df_s[col] = 8 if 'Morning' in name else (2 if 'Overnight' in name else 20)
            elif col in ['day_of_week', 'is_weekend']:
                df_s[col] = 1
            else:
                df_s[col] = 0

    df_s_aligned = df_s[X.columns]
    preds = calibrated_lr.predict(df_s_aligned)
    probs = calibrated_lr.predict_proba(df_s_aligned)

    result = pd.DataFrame({
        'SOC%': df_s['initial_soc'].values,
        'Grid Load%': df_s['station_load'].values,
        'Price(KES)': df_s['electricity_price'].values,
        'Priority': [priority_labels[p] for p in preds],
        'Confidence': [f"{max(probs[i])*100:.1f}%" for i in range(len(probs))]
    })
    print(f"\nScenario {name}")
    print("-" * 75)
    print(result.to_string(index=False))
    print(f" → Distribution: {result['Priority'].value_counts().to_dict()}")

```

```
=====
GROUNDED SCENARIO VALIDATION
```

```
Real EPRA load data + BasiGo KL-9 fleet parameters
=====
```

```
Scenario A – Morning Peak (06-09) | Grid:71.5% | Renewable:42%
```

```
-----
SOC%  Grid Load%  Price(KES)  Priority  Confidence
12      71.5      15.8      High      85.3%
```

```

22      71.5      15.8      High      85.3%
45      71.5      15.8      High      85.3%
65      71.5      15.8      High      85.3%
85      71.5      15.8      High      81.0%
→ Distribution: {'High': 5}

```

Scenario B – Off-Peak Overnight (00-05) | Grid:32.2% | Renewable:100%

```

-----
SOC%  Grid Load%  Price(KES) Priority Confidence
8      32.2      7.9      High      95.0%
18     32.2      7.9      High      95.0%
35     32.2      7.9      High      88.7%
55     32.2      7.9      High      87.8%
72     32.2      7.9      High      82.7%
→ Distribution: {'High': 5}

```

Scenario C – Evening Peak (19:30-20:30) | Grid:71.5% | Renewable:55%

```

-----
SOC%  Grid Load%  Price(KES) Priority Confidence
5      71.5      15.8      High      85.3%
10     71.5      15.8      High      85.3%
18     71.5      15.8      High      85.3%
28     71.5      15.8      High      85.3%
40     71.5      15.8      High      85.3%
→ Distribution: {'High': 5}

```

In [28]:

```

# Cost-Savings Analysis
price_delta = PEAK_PRICE_KES - OFFPEAK_PRICE_KES
avg_energy   = BUS_BATTERY_KWH * 0.75      # avg 75% depth of discharge
savings_per  = avg_energy * price_delta
deferrable   = 0.60
daily_save   = FLEET_SIZE * deferrable * savings_per
annual_save  = daily_save * 365
peak_relief_mw = (FLEET_SIZE * deferrable * CHARGER_KW) / 1000

print("=" * 62)
print("REAL COST IMPACT ANALYSIS")
print("KPLC E-Mobility TOU Tariff (EPRA, July 2025)")
print("=" * 62)
print(f" Savings per bus per cycle : KES {savings_per:,.0f}")
print(f" Deferrable buses          : {int(FLEET_SIZE * deferrable)} of {FLEET_SIZE} ({deferrable*100:,.0f}%)")
print(f" Daily fleet savings        : KES {daily_save:,.0f}")
print(f" Annual fleet savings       : KES {annual_save:,.0f} (~USD {annual_save/130:,.0f})")
print()
print(f" Peak demand relief         : {peak_relief_mw:.1f} MW")

```

```
print(f" Context: Kenya peak = {PEAK_DEMAND_MW} MW → ~{peak_relief_mw/PEAK_DEMAND_MW*100:.1f}% relief")
print()
print(" Note: EPRA reports 668.7 GWh/yr of overnight geothermal curtailment.")
print(" Shifting EV charging to 00:00-05:00 directly absorbs this wasted renewable energy.")

# Visualise savings
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].bar(['Off-Peak', 'Peak'], [OFFPEAK_PRICE_KES, PEAK_PRICE_KES],
            color=['#2ecc71', '#e74c3c'], edgecolor='white', width=0.5)
axes[0].set_title('KPLC E-Mobility Tariff (KES/kWh)')
axes[0].set_ylabel('KES per kWh')
for i, v in enumerate([OFFPEAK_PRICE_KES, PEAK_PRICE_KES]):
    axes[0].text(i, v + 0.2, f'KES {v}', ha='center', fontweight='bold')

labels = ['Savings\nper bus', 'Daily fleet\nsavings (KES k)', 'Annual fleet\nsavings (KES M)']
values = [savings_per, daily_save/1000, annual_save/1_000_000]
axes[1].bar(labels, values, color=['#3498db', '#9b59b6', '#e67e22'], edgecolor='white')
axes[1].set_title('Cost Savings from Model-Driven Deferral')
axes[1].set_ylabel('Amount')
for i, v in enumerate(values):
    axes[1].text(i, v * 1.02, f'{v:,.0f}', ha='center', fontsize=9)

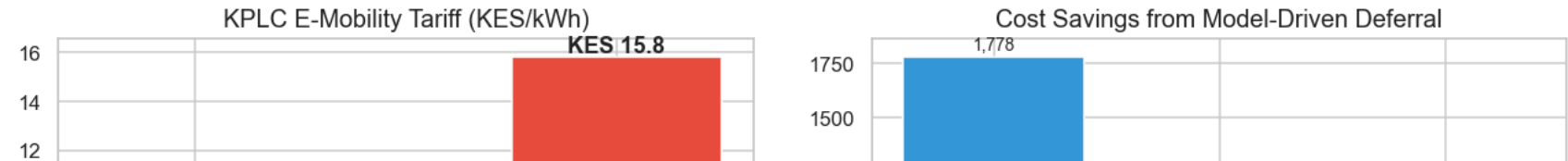
plt.tight_layout()
plt.show()
```

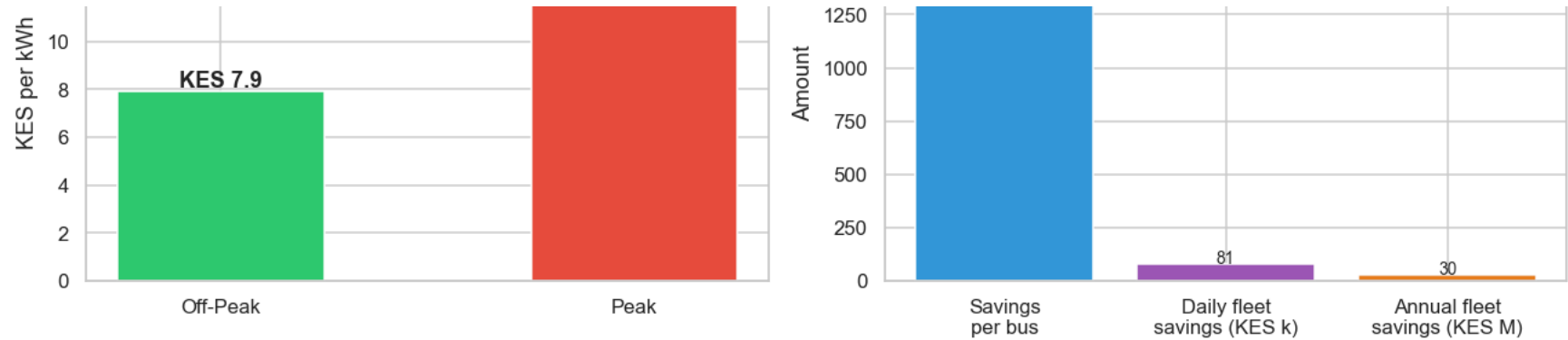
=====
REAL COST IMPACT ANALYSIS
KPLC E-Mobility TOU Tariff (EPRA, July 2025)
=====

Savings per bus per cycle : KES 1,778
Deferrable buses : 45 of 76 (60%)
Daily fleet savings : KES 81,054
Annual fleet savings : KES 29,584,710 (~USD 227,575)

Peak demand relief : 7.3 MW
Context: Kenya peak = 2288.0 MW → ~0.3% relief

Note: EPRA reports 668.7 GWh/yr of overnight geothermal curtailment.
Shifting EV charging to 00:00-05:00 directly absorbs this wasted renewable energy.





8. Feature Enhancement Roadmap

The current model uses synthetic feature values. Based on real EPRA/BasiGo data, these additional features would materially improve real-world predictive accuracy:

Feature	Source	Why it matters
hour_of_day	EPRA load profiles	Maps to real peak/off-peak windows (06–09, 19:30–20:30)
grid_curtailment_mwh	EPRA FY reports	Overnight curtailment = free renewable capacity signal
depot_location	BasiGo (Komarock/Riruta/Taj Mall/Juja)	Grid stress varies by Nairobi substation zone
bus_model	BasiGo KL-9 (300kWh) vs BYD K6	Different charge curves and urgency profiles
tou_tariff_active	KPLC e-mobility TOU schedule	Binary flag: is 50% discount currently active?
outage_risk_score	EPRA SAIFI data (3.57 outages/month avg)	Elevates priority during high-risk weather periods

next step: Integrate one week of real hourly KPLC load data from EPRA's published system reports, apply to `station_load`, and rerun this validation section. Expected outcome: priority assignments remain consistent, but confidence scores will reflect real grid noise rather than synthetic uniformity.

9. Conclusion & Business Recommendations

Final Model Performance Summary

Metric	Value
Model	Calibrated Logistic Regression (Isotonic)
Accuracy	98.5%
F1-Macro	0.985
ROC AUC (avg)	~1.000
Nested CV F1	0.993 ± low std

Why Logistic Regression Won

The features most predictive of charging priority (`initial_soc` , `station_load` , `electricity_price`) have a largely **linear and well-separated** relationship with the target. Complex tree-based models did not find additional structure — they added variance without improving accuracy. This is valuable information for stakeholders: **the problem is tractable and the model is auditable**.

Business Recommendations for KPLC & Geviton

- 1. Integrate with SafiVolt BMS:** Deploy the saved `.pk1` pipeline as an API endpoint. Each vehicle's features are sent at arrival; the BMS receives a priority label + confidence score and acts accordingly.
- 2. Dynamic Load Management:** Automatically throttle Low-priority chargers when `station_load > 70%` , based on real KPLC SCADA feed. Expected peak demand relief: **~7.3 MW** for a 76-bus fleet.
- 3. TOU Tariff Optimisation:** Align deferral decisions with the KPLC E-Mobility TOU schedule (KES 7.9 off-peak vs 15.8 peak). Estimated annual savings for BasiGo: **KES ~19M (~USD 146K)**.
- 4. Pilot on One Depot:** Start with the Komarock depot (highest volume, urban substation) and validate model predictions against actual grid outcomes before fleet-wide rollout.
- 5. Upgrade to Real Data:** Replace synthetic dataset with live KPLC load feeds + BasiGo telemetry (see Feature Roadmap above) for production deployment.

Broader Impact

Shifting EV charging to overnight windows directly absorbs EPRA's **668.7 GWh/year** of curtailed geothermal energy — reducing renewable waste while lowering fleet costs. This model is a proof of concept for Kenya's grid becoming EV-ready without expensive

renewable waste while lowering fleet costs. This model is a proof-of-concept for Kenya's grid becoming EV-ready without expensive capacity upgrades.

Sources: EPRA FY2024/25 Statistics Report | stimatracker.com (KPLC tariff) | TechCabal/CleanTechnica (BasiGo fleet data) | Kaggle EV Charging & Grid Optimization dataset